Hurricane Katrina Review -Insurance

Import Variables

In [7]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Bring in dataset

In [8]:
```
roadhome = pd.read_csv(
    "https://raw.githubusercontent.com/wujenny214/Policy-Study-into-Grant-Assistance-for-Post-Hurricane-Reb
)
```

```
/var/folders/xm/yhmrwng57qq9wlcywlm89pxh0000gn/T/ipykernel_32129/320429160.py:1: DtypeWarning: Columns (1
3,15) have mixed types. Specify dtype option on import or set low_memory=False.
  roadhome = pd.read_csv(
```

Clean data

In [9]:
```
roadhome = roadhome[roadhome["GIS State"] == "LA"]
roadhome = roadhome[roadhome["ARS File (Yes/No)"] == "N"]
roadhome = roadhome[roadhome["Current Damage Assessment"] != 9999999.0]
```

In [10]:
```
roadhome
```

Out[10]:

	Unnamed: 0	Structure Type	GIS City	GIS State	GIS Zip	PARISH	Closing Option	TOTAL_CLOSING_AMOUNT	Total CG Amount
0	9613.0	Single (including mobile home)	METAIRIE	LA	70001	Jefferson	1.0	150000.00	95213.27
1	21349.0	Duplex (with one owner-occupied unit)	METAIRIE	LA	70001	Jefferson	2.0	150000.00	150000.00
2	25325.0	Duplex (with one owner-occupied unit)	METAIRIE	LA	70001	Jefferson	1.0	3450.14	3450.14
3	27236.0	Duplex (with one owner-occupied unit)	METAIRIE	LA	70001	Jefferson	1.0	10665.59	10665.59
4	36177.0	Single (including mobile home)	METAIRIE	LA	70001	Jefferson	1.0	131947.98	101947.98
...	...	...	...	...	...	...	...	...	...
130048	44583.0	Mobile Home	OAKDALE	LA	71463-6000	Allen	1.0	26356.16	6997.46
130049	85246.0	Mobile Home	OAKDALE	LA	71463-8005	Evangeline	1.0	9015.20	9015.20
130050	121180.0	Single (including mobile home)	ZWOLLE	LA	71486-2603	Sabine	1.0	14802.42	14802.42
130051	124807.0	Single (including mobile home)	ZWOLLE	LA	71486-2761	Sabine	1.0	14471.91	14471.91
130052	97713.0	Mobile Home on Leased Land	ZWOLLE	LA	71486-3562	Sabine	1.0	9411.57	9411.57

121082 rows × 37 columns

Summarize the Pre-Storm value

In [11]:
```
roadhome["Current PSV"].describe()
```

Out[11]:
```
count    1.210820e+05
mean     1.365587e+05
std      9.773124e+04
min      1.000000e+00
25%      8.500000e+04
50%      1.220000e+05
75%      1.640000e+05
max      3.400000e+06
Name: Current PSV, dtype: float64
```

Checking the property value by quantile to identify lower-income residents, and associate with property

In [12]:
```
roadhome["Current PSV"].quantile(0.85)
```

Out[12]:
```
203471.6849999999
```

Bin by property value to illustrate disparity in insurance

In [13]:
```
bins = [0, 40000, 80000, 120000, 160000, 200000, 240000, float("inf")]
labels = [
    "<$40K",
    "$40–80K",
    "$80–120K",
    "$120–160K",
    "$160–200K",
    "$200–240K",
    "$240K+",
]
```

In [14]:
```
roadhome["property_bucket"] = pd.cut(
    roadhome["Current PSV"], bins=bins, labels=labels, include_lowest=True
)
```

Segment by bucket

In [15]:
```
roadhome["property_bucket"].value_counts()
```

Out[15]:
```
property_bucket
$80–120K     31904
$120–160K    30461
$40–80K      14986
$160–200K    12822
<$40K        12356
$240K+       12063
$200–240K     6490
Name: count, dtype: int64
```

Recode Insurance holders to represent "Y" with "1" and "N" with "0"

In [16]:
```
roadhome["insurance_num"] = roadhome[
    "Applicant With\
 Current Insurance (Private and/or Flood) Y/N\
"
].map({"Y": 1, "N": 0})
```

Prepare table to compute proportion of people with insurance by property value group

In [17]:
```
avg_response = roadhome.groupby("property_bucket")["insurance_num"].mean().reset_index()

print(avg_response)
```

```
  property_bucket  insurance_num
0           <$40K       0.277355
1         $40–80K       0.640598
2        $80–120K       0.829081
3       $120–160K       0.906930
4       $160–200K       0.919825
5       $200–240K       0.928968
6          $240K+       0.938821
```

```
/var/folders/xm/yhmrwng57qq9wlcywlm89pxh0000gn/T/ipykernel_32129/2331860686.py:1: FutureWarning: The defaul
t of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=
False to retain current behavior or observed=True to adopt the future default and silence this warning.
  avg_response = roadhome.groupby("property_bucket")["insurance_num"].mean().reset_index()
```

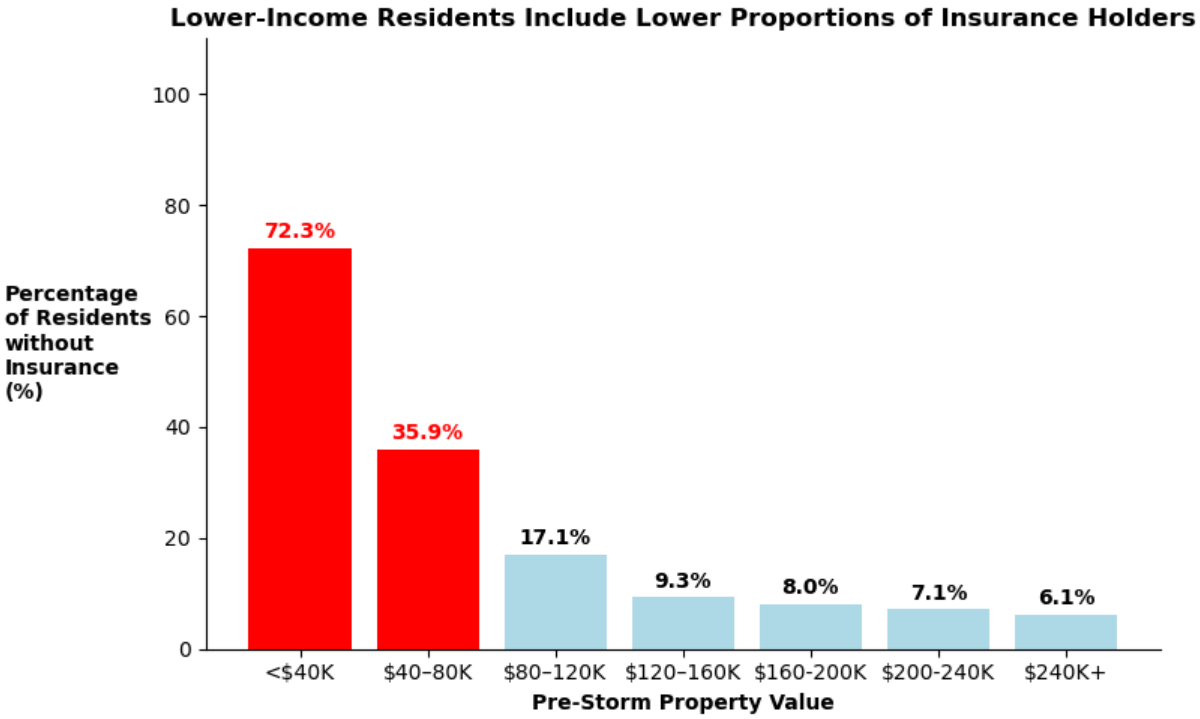
In [18]:
```
count_response = (
    roadhome.groupby("property_bucket")["insurance_num"].count().reset_index()
)

print(count_response)
```

```
  property_bucket  insurance_num
0           <$40K          12356
1         $40–80K          14986
2        $80–120K          31904
3       $120–160K          30461
4       $160–200K          12822
5       $200–240K           6490
6          $240K+          12063
/var/folders/xm/yhmrwng57qq9wlcywlm89pxh0000gn/T/ipykernel_32129/1853335242.py:2: FutureWarning: The defaul
t of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=
False to retain current behavior or observed=True to adopt the future default and silence this warning.
  roadhome.groupby("property_bucket")["insurance_num"].count().reset_index()
```

Graph chart showing insurance non-holders by property value bracket

In [19]:
```
# Graph chart
plt.figure(figsize=(8, 5))
bar_colors = ["red" if i < 2 else "lightblue" for i in range(len(avg_response))]

# Graph chart
plt.figure(figsize=(8, 5))
bars = plt.bar(
    avg_response["property_bucket"],
    (1 - avg_response["insurance_num"]) * 100,
    color=bar_colors,  # updated here
)


# Assign axis labels
plt.xlabel("Pre-Storm Property Value", fontweight="bold")
plt.ylabel(
    "Percentage\nof Residents\nwithout\nInsurance\n(%)",
    fontweight="bold",
    rotation=0,
    labelpad=70,
    va="center",
    ha="left",
)
plt.title(
    "Lower-Income Residents Include Lower Proportions of Insurance Holders",
    fontweight="bold",
)
plt.ylim(0, 110)

# Prepare bar labels
for i, bar in enumerate(bars):
    height = bar.get_height()
    label_color = "red" if (i == 0 or i == 1) else "black"

    plt.text(
        bar.get_x() + bar.get_width() / 2,
        height + 1,
        f"{height:.1f}%",
        ha="center",
        va="bottom",
        fontweight="bold",
        color=label_color,
    )

sns.despine()

plt.tight_layout()
plt.show()
```

```
<Figure size 800x500 with 0 Axes>
```



Compute Insurance Duplicative Benefits

Compute duplicative benefits and its proportion of house value

In [20]:
```
roadhome["DOB Proportion"] = (
    roadhome["Closing Total DOB Amount"] / roadhome["Current PSV"]
) * 100
```

Take mean of insurance holders and non-insurance holders

In [21]:
```
avg_dup_ben = roadhome.groupby("insurance_num")["DOB Proportion"].mean().reset_index()
```

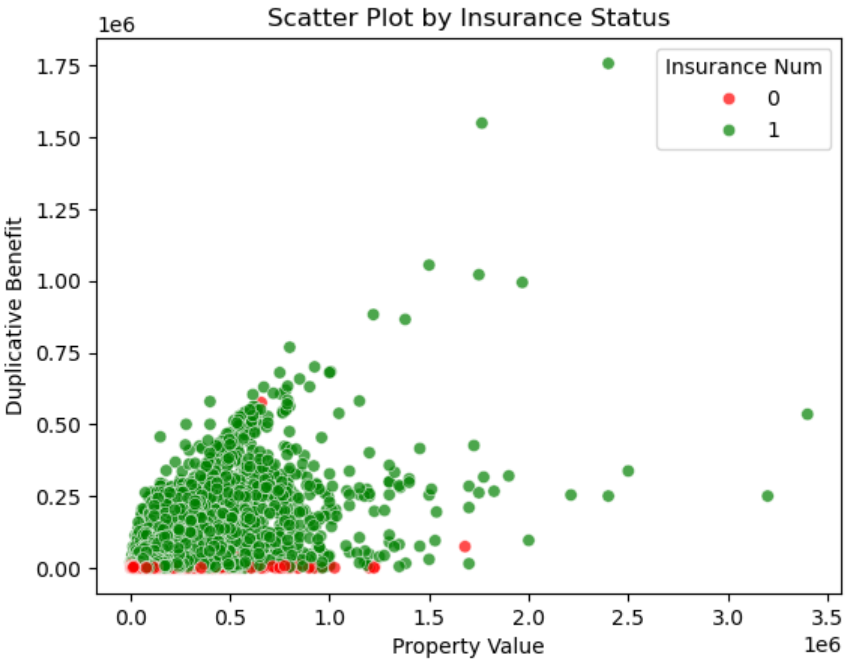
In [22]:
```
avg_dup_ben
```

Out[22]:

	insurance_num	DOB Proportion
0	0	13.535602
1	1	58.631788

Scatter Plot of Insurance and Non-insurance holders

In [23]:
```
# Scatter Plot
sns.scatterplot(
    data=roadhome,
    x="Current PSV",
    y="Closing Total DOB Amount",
    hue="insurance_num",
    palette={0: "red", 1: "green"},
    alpha=0.7,
)

plt.xlabel("Property Value")
plt.ylabel("Duplicative Benefit")
plt.title("Scatter Plot by Insurance Status")
plt.legend(title="Insurance Num")
plt.show()
```


Scatter Plot by Insurance Status
1e6
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
Duplicative Benefit
Insurance Num
0
1
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
Property Value
1e6